

Horus Knowledge Transfer & Open-Source Transition

Architecture Overview

Horus is a multi-platform, node-based scientific workflow designer. The stack is split into:

- **Python 3.9 backend** (`App/`, `Server/`, `HorusAPI/`) — Flask + Flask-SocketIO server, process-per-block execution model, HPC/SLURM integration via SSH, four start-up modes.
- **TypeScript/React frontend** (`Views/`, compiled to `GUI/`) — Parcel-bundled SPA with Mol* molecular viewer, SMILES editor (JSME), real-time SocketIO comms, TailwindCSS.
- **HorusAPI Python package** (`HorusAPI/`) — the block/plugin contract shared between the app and plugin authors; compiled and published to PyPI.
- **AppSupport directory** — runtime data: plugins, flows, config, logs, templates, SLURM scripts.
- **CI/CD** (`.github/workflows/ci.yml`) — self-hosted runner; on any tag: tests + lint → Docker/Singularity build → PyPI publish → Sphinx docs rebuild. The four **run modes** are: `app` (PyWebview desktop), `browser` (headless with system browser), `server` (plain Flask), `webapp` (multi-user, Flask-Login, SQLite). A **flow** is a JSON `.flow` file. Each block in a running flow becomes an **independent subprocess** launched with `Horus.py --flow <path> --index <placedID>`. SLURM blocks additionally submit Bash scripts to an HPC cluster over SSH (via `fabric`).

Session Syllabus

Part I — Foundations

Session 1: Repository Tour & Development Environment (2 h)

Objectives: Orient every team member in the repository, get the dev environment running, and understand the overall layered architecture. **Concepts:** Monorepo layout; Yarn Berry PnP; Micromamba conda env; `bun run requirements` bootstrap; `python Horus.py --debug --server` vs `--app` vs `--browser`; `APP_INFO` version file; `HORUS_APP_SUPPORT_DIR` and other env-var overrides. **Core modules / files:**

- Root tooling: `package.json`, `.parcelrc`, `.postcssrc`, `tailwind.config.js`, `tsconfig.json`, `Devtools/Environment/conda_horus.yml`
- Entry point: `Horus.py` → `App/__init__.py` → `App/app_delegate.py:launchApp()`
- AppSupport layout: `AppSupport/Plugins/`, `DefaultPlugins/`, `config/`, `custom_blocks/`, `logs/` **Integration points:** The `HORUS_APP_SUPPORT_DIR`, `HORUS_PORT`, `HORUS_HOST`, `HORUS_MODE` environment variables that control deployment. `App/APP_INFO` file tied to `HorusAPI.__version__`. **Hands-on:** Clone, run `bun run requirements`, activate env, launch `python Horus.py --debug --server`, navigate to `http://127.0.0.1:3000`.

Session 2: Backend App Lifecycle — `AppDelegate` & Launch Modes (2 h)

Objectives: Trace exactly what happens from `launchApp()` through server startup to the first browser request, understand all four modes, and be able to add new CLI flags. **Concepts:** `AppDelegate` singleton pattern (`HorusSingleton`); `HorusLogger` + `FakeWriter` (stdout/stderr capture to file);

`WindowOptions`; `webview.token` security token; `parseArgs()` parser; `runFlowInsteadOfLaunch()` and `installPluginInsteadOfLaunch()` fast-paths; `debugpy` remote debugger attachment. **Core modules / files:**

- `App/app_delegate.py` — `AppDelegate.__init__`, `initializeServer`, `applicationDidFinishLaunching`, `_startAppMode`, `_startServerMode`, `_startBrowserMode`
- `Server/Utils.py` — `PrintTruncator` **Integration points:** `AppDelegate` is the single authority on `appSupportDir` and server instance. Any new mode or startup flag starts here. The `FLOWPATH` module-level global is how block subprocess invocations communicate back their log path. **Hands-on:** Add a `--dry-run` CLI flag that prints the resolved config and exits without starting the server.

Session 3: The Flask Server — `HorusServer` Init, Routes & Middleware (3 h)

Objectives: Understand how Flask is configured, all route groups, all decorator middlewares, and how to add a new API endpoint safely. **Concepts:** `HorusFlask` subclass (root-path injection); CORS setup via `flask-cors`; `Flask-Login` integration; `TokenManager`; route decorator middlewares (`verifyLogin`, `stopDemoUser`, `verifyQuotas`, `noWebApp`, `allowRemotes`, `preventOnWebApp`); `socketio.emit("flow", ...)` real-time push; error handler chain; `_pluginPages()` for per-plugin static HTML pages. **Core modules / files:**

- `Server/server.py` — `HorusServer.__init__`, `_setupServer`, `_setupCORS`, `_setupLoginManager`, `_routes()`, `_userRoutes()`, `_setupSocketio()`, `_pluginPages()`, `_exceptionHandlers()` **Key API routes (from `_routes()`):** `POST /api/saveflow`, `GET /api/getflow`, `POST /api/runflow`, `POST /api/stopflow`, `GET /api/getblocks`, `GET /api/settings`, `POST /api/saveremote`, `GET /api/fileexplorer`, etc. **Key SocketIO events:** `"flow"` (block status push), `"print"` (block stdout), `"slurm"` (job status). **Integration points:** Every new feature that the frontend needs must register a route here, behind the appropriate middlewares. The `_isForUser` property switches between global and per-user data managers in webapp mode. **Hands-on:** Register a `GET /api/version` endpoint that returns `{"version": HorusAPI.__version__, "mode": self.mode}` and verify it from the browser.

Part II — The Plugin API (HorusAPI)

Session 4: Block & Variable Model (3 h)

Objectives: Be able to read, author, and debug any plugin; understand every variable type and how inputs/outputs flow between blocks at execution time. **Concepts:** `Plugin` class; `plugin.meta` file format; `VariableTypes` enum (30+ types); `PluginVariable`, `VariableGroup`, `VariableList`, `CustomVariable`; `PluginBlock` lifecycle (`__init__`, `__call__`, `_parseInternalVariables`, `_minimalEncode`); `setOutput`; `blockType` enum (`BASE`, `INPUT`, `ACTION`, `SLURM`, `CONFIG`, `GHOST`); `InputBlock`, `PluginConfig`, `GhostBlock`; `unique_block_dir_local/remote`; `block.flow`, `block.config`, `block.extraData`. **Core modules / files:**

- `HorusAPI/src/plugins.py` — all classes above
- `HorusAPI/src/molstar.py` — Mol* helper API
- `HorusAPI/src/smiles.py` — SMILES helper

- `HorusAPI/src/extensions.py` — `Extensions` API for triggering UI panels
 - `AppSupport/DefaultPlugins/Horus/Horus.py` — the built-in plugin as a worked example
- Integration points:** `block.flow` gives access to other blocks, flow path, and `write()`. `block.config` accesses `PluginConfig` values. `setOutput(id, value)` populates downstream block inputs. **Hands-on:** Write a minimal `ActionBlock` that takes a `FILE` input, counts lines, and outputs an `INTEGER`; install it as a dev plugin and run it in a flow.
-

Session 5: SlurmBlock, Remote Execution & Custom Blocks (3 h)

Objectives: Know how to write blocks that submit jobs to an HPC cluster, understand the SSH data-transfer model, and maintain the custom-block JSON editor. **Concepts:** `SlurmBlock` base class; `PluginRemote` facade (`command`, `sendData`, `getData`, `submitJob`, `cd` context manager); `SlurmJob` Pydantic model and `Status` enum; `finalAction` callback; `continueSlurm` and `resetRemoteBlock` restart flows; custom-block JSON schema (`AppSupport/custom_blocks/*.json`); `CustomBlockParser`. **Core modules / files:**

- `HorusAPI/src/plugins.py` — `SlurmBlock`, `PluginRemote`, `SlurmJob`, `Status`
 - `Server/RemotesManager/remotes_manager.py` — `RemotesAPI.connect`, `command`, `transferTo`, `transferFrom`
 - `AppSupport/custom_blocks/slurm.json` — example custom SLURM block
- Integration points:** `PluginRemote.submitJob()` wraps `SlurmJob._submitJob()` which calls `sbatch` and parses `scontrol` output. The `FlowManager` polls job status and re-triggers `finalAction` when SLURM jobs complete. **Hands-on:** Write a `SlurmBlock` that submits a simple `echo` job on a remote and reads back the output file.
-

Part III — The Flow Engine

Session 6: Flow Persistence — The `.flow` Format & `FlowManager` (2 h)

Objectives: Understand how flows are serialised/deserialised, how the `.flow` JSON is structured, how `FlowManager` handles create/open/save/template operations. **Concepts:** Flow JSON anatomy (`name`, `path`, `savedID`, `blocks[]`, `connections[]`, `smilesState`); `Flow.encode()` vs `_minimalEncode()`; `FlowManager.saveFlow()`, `openFlowFromPath()`, `saveAsTemplate()`, `recentFlows`; `OverwriteException` guard; flow working directory (`Flow.flowWorkDir()`); `BlockConnection` and `BlockVarPair` serialisation. **Core modules / files:**

- `Server/FlowManager/flow_manager.py` — `Flow`, `FlowManager`, `OverwriteException`, `TemplateNotFound`
 - `AppSupport/recent_flows.json` — recent flows
 - `AppSupport/templates/` — template flows
 - `Tests/TestFlows/test_flow.flow` — test fixture
- Integration points:** `POST /api/saveflow` → `FlowManager.saveFlow()` → `socketio.emit("flow", ...)`. Opening a flow triggers plugin lookup via `PluginManager.getBlock()`, replacing unknown blocks with `GhostBlock`. **Hands-on:** Open `Tests/TestFlows/test_flow.flow` in a Python shell, read its block states, and manually trigger `flow.write()`.
-

Session 7: Flow Execution — Subprocess Model, SLURM Polling & Cyclic Flows (3 h)

Objectives: Trace a complete flow run from the frontend **Run** button through subprocess spawning, socket status updates, error handling, and SLURM polling; understand cyclic (looping) flows. **Concepts:**

Flow.run(placedID, ...) entry point; DAG traversal with dependency resolution; subprocess launch **Horus.py --flow <path> --index <id> --flow-base-url <url>**; **FlowRunInfo** (PID/SLURM env detection); **socketio** rooms keyed by **flow.savedID**; **StoppedFlowException**; **VariableConnectionNotFound**; cyclic connections (**isCyclic**, **cycles**, **currentCycle**); **pauseAllFlows()** on app exit; **resetFlow**, **continueSlurm** flags. **Core modules / files:**

- **Server/FlowManager/flow_manager.py** — **Flow.run**, **FlowManager.areThereRunningFlows**, **pauseAllFlows**
- **Server/PluginManager/plugin_manager.py** — **SubprocessManager**, **PrintCapturer**
- **App/app_delegate.py** — **runFlowInsteadOfLaunch** **Integration points:** Block subprocess inherits **HORUS_APP_SUPPORT_DIR** and **--flow-base-url** to POST status events back to the parent server. The parent server receives these via **POST /api/blockevent** and broadcasts via **SocketIO**. **Hands-on:** Enable **DEBUG** logging and run **Tests/TestFlows/test_flow.flow** end-to-end; trace the subprocess PID in the logs and correlate it to the **SocketIO "print"** events in the browser console.

Part IV — Server Managers Deep-Dive

Session 8: **PluginManager** — Discovery, Loading & Hot-Reload (2 h)

Objectives: Know how plugins are discovered, loaded, and hot-reloaded; be able to add a new plugin repo, install from CLI, and debug missing-dependency errors. **Concepts:** **AppSupport/Plugins/** and **DefaultPlugins/** scan; **plugin.meta** parsing (**PluginMetaModel**); **importlib.util** dynamic loading; **PluginDeps** context manager (isolated **sys.path** per plugin); **pluginChanges** dirty flag; **REPOIDURL_PREFIX**, **PLUGINIDURL_PREFIX**, **REPONAME_PREFIX** URL schemes for **--install-plugin**; **ExternalPython**; **MissingDependencies** reporting; **PluginRepos**. **Core modules / files:**

- **Server/PluginManager/plugin_manager.py** — **PluginManager.__init__**, **_initializePlugins**, **_loopPluginsToInstall**, **getBlock**, **ExternalPython**
- **AppSupport/DefaultPlugins/Horus/plugin.meta** — canonical meta file example
- **Tests/TestPluginManager/** — test plugin fixture **Integration points:** **PluginManager** is a **HorusSingleton**; **HorusServer.pluginManager** accesses it. **GET /api/getblocks** returns **pluginManager.getBlocks()**. **POST /api/installplugin** triggers **_loopPluginsToInstall**. **Hands-on:** Use **--install-plugin** to install the dev plugin from **Tests/TestPluginManager/Plugins/** and verify the block appears in **GET /api/getblocks**.

Session 9: **RemotesManager** — SSH, File Transfer & Proxy (2 h)

Objectives: Configure and debug SSH remote connections, understand how files move to/from HPC, and add new connection options. **Concepts:** **RemotesAPI** (Fabric wrapper); **connect()**; **command(timeout, mergeStdErr, forceLocal, env)**; **transferTo/From** (rsync/SCP under the hood); **cd** context manager; **proxyCommand** jump-host; **loadProfile** toggle; **workDir** on remote;

`RemotesManager.addRemote/removeRemote`; `remotes.json` persistence; `ConnectionFailed` / `CommandFailed` exceptions. **Core modules / files:**

- `Server/RemotesManager/remotes_manager.py` — `RemotesAPI`, `RemotesManager`
 - `AppSupport/remotes.json` — persisted remote definitions
 - `Views/Remotes/` — frontend remote configuration UI **Integration points:** `GET /api/getremotes`, `POST /api/saveremote`, `POST /api/testremote`, `DELETE /api/deleteremote`. The `PluginRemote` facade in `HorusAPI/src/plugins.py` wraps `RemotesAPI` and is the only interface blocks should use. **Hands-on:** Manually add a `localhost` remote (using SSH to `127.0.0.1`) via `remotes.json`, test it with `POST /api/testremote`, and run a block that executes `hostname` on it.
-

Session 10: `WebAppManager` — Multi-User Mode, Auth & Quotas (2 h)

Objectives: Deploy and configure Horus in `--webapp` mode with full user registration; understand the `horus.config.json` schema, quotas, and the admin panel. **Concepts:** `horus.config.json` structure (`host`, `port`, `appName`, `cors`, `userManagement`, `allowRemotes`); `DatabaseConfig` (SQLite path + secret key); `DefaultQuotas` (`maxStorage`, `maxFlows`, `maxTemplates`, `maxTime`, `resetTime`); `ExtraField` registration form fields; `HorusUser` (`flask_login.UserMixin`); demo user vs anonymous user vs registered user; `APScheduler` for quota reset; `verifyQuotas` decorator; `UserFileExplorer` path sandboxing. **Core modules / files:**

- `Server/WebAppManager/webapp_manager.py` — `WebAppManager`, `DefaultQuotas`, `DatabaseConfig`
 - `Server/WebAppManager/database.py` — `Database`, `UserError`
 - `Server/WebAppManager/user.py` — `HorusUser`
 - `horus.config.json` (root) — example configuration **Integration points:** On `mode == "webapp"`, `HorusServer` replaces global manager instances with per-user instances keyed by `currentUser.appSupportDir`. `_userRoutes()` adds `/users/login`, `/users/register`, `/users/reset`, `/users/admin`. **Hands-on:** Copy `horus.config.json`, set `requireRegistration: true`, launch in `--webapp` mode, register a user, and verify their `horus_users/` folder is created.
-

Part V — Frontend

Session 11: Frontend Architecture — Views, Components & Build System (3 h)

Objectives: Understand how the React/TypeScript UI is structured, how Parcel builds it, and how to add a new view or component. **Concepts:** `Views/` source → `GUI/` compiled; per-page entry points (`Views/Main/index.html`, `Views/Login/login.html`, etc.); Parcel config (`.parcelrc`); Yarn Berry PnP; TailwindCSS (`tailwind.config.js`, PostCSS); TypeScript strict mode (`tsconfig.json`); shared component patterns (`Views/Components/`); `globals.ts`, `utils.ts`, `navigationService.ts`; `Views/Utils/socket.ts` (Socket.IO client). **Key component families:**

- `FlowBuilder/` — `flow.view.tsx`, `flow.hooks.ts`, `flow.types.ts` (canvas, block drag-drop)
- `Molstar/molstar.tsx` — Mol* 3-D viewer integration
- `Smiles/` — JSME 2-D editor + `SmilesGrid/SmilesViewport`

- `Toolbar/toolbar.tsx` — top action bar
 - `Console/console.tsx` — live block stdout stream
 - `FlowStatus/` — run status indicator
 - `FileExplorer/file_explorer.tsx` — server-side file picker **Core modules / files:**
 - `Views/Main/app.tsx`, `Router.tsx`, `index.tsx`
 - `Views/Components/FlowBuilder/flow.view.tsx`
 - `Views/Utils/socket.ts` **Integration points:** `Views/Utils/socket.ts` connects to the Flask-SocketIO server and broadcasts SocketIO events (`"flow"`, `"print"`, `"slurm"`) to subscribed components. REST calls go through `utils.ts` fetch wrappers that attach the `shemsu` auth token.
- Hands-on:** `bun run dev` (Parcel watch mode), add a small info badge to `toolbar.tsx` that calls `GET /api/version` and displays the version string.
-

Session 12: Frontend–Backend Integration — SocketIO, REST & Flow State (2 h)

Objectives: Understand the full data-flow for a block run (REST trigger → subprocess → SocketIO status push → React state update → UI), and know how to add a new real-time event. **Concepts:** `shemsu` token query-param security; SocketIO rooms (`join_room(flow.savedID)`); event schema for `"flow"`, `"print"`, `"slurm"`; `useContainerSize` hook; `flow.hooks.ts` (React state for block statuses); `HorusPlot` (Plotly integration); `HorusLazyLog` (lazy log streaming); extension API (`Extensions.openExtension()` → `"openExtension"` SocketIO event). **Core modules / files:**

- `Server/server.py` — `_setupSocketio()`, `socketio.emit("flow", ...)`, `socketio.emit("print", ...)`
 - `Views/Utils/socket.ts` — `onFlow`, `onPrint`, `onSlurm` listeners
 - `Views/Components/FlowBuilder/flow.hooks.ts`
 - `HorusAPI/src/extensions.py` — `Extensions` API **Integration points:** When a block finishes, the subprocess posts to `POST /api/blockevent`; the server updates the `Flow` object and emits `socketio.emit("flow", flow.encode(minimal=False), to=flow.savedID)`. The frontend room joined at `flow-open` receives this and updates the canvas. **Hands-on:** Add a new `"heartbeat"` SocketIO event emitted every 30 s from the server and log it in the browser console.
-

Part VI — DevOps & Open-Source Transition

Session 13: Build, Package & Distribution (2 h)

Objectives: Be able to produce a release independently: compile the frontend, bundle the Python app with PyInstaller, build Docker/Singularity images, create `.deb/.rpm/.dmg` packages, and publish `HorusAPI` to PyPI. **Concepts:** `bun run build` → `bun run buildparcel` then PyInstaller; `Devtools/Compile/compile.py`, `build.spec`, `macos-hook.py`; `Devtools/Package/` scripts per distro; Docker targets (`Devtools/Docker/`); Singularity multi-platform build (`Devtools/Singularity/`); `HORUS_VERSION_OVERRIDE`; `HorusAPI` Cython compilation (`build_horusapi.sh`); `auditwheel` manylinux wheel repair; `twine upload`. **Core modules / files:**

- `Devtools/Compile/build.spec` — PyInstaller spec
- `Devtools/Docker/horus.dockerfile`, `centos7.dockerfile`, `ubuntu22.dockerfile`, `rocky8.dockerfile`

- `.github/workflows/ci.yml` — full CI pipeline
- `Devtools/build_docs.sh` — Sphinx docs rebuild **Integration points:** CI is tag-triggered; `test-and-lint` runs on every PR/push. The `publish_horusapi` job uses the manylinux Docker image to produce a CPython 3.9 wheel; `build_horus` produces container images. All artefacts attach to the GitHub Release. **Hands-on:** Run `bun run build` locally, inspect the `dist/` output, and manually run `python Devtools/Compile/compile.py` on a branch. **Known gap to fix:** CI runner `perry` is a private self-hosted machine. Migrating to portable GitHub-hosted runners is required before the open-source launch (see roadmap).

Session 14: Testing, Linting & Quality Gates (1.5 h)

Objectives: Run the full test suite, understand each test domain, add a new test, and enforce quality gates in CI. **Concepts:** pytest; test domains: `TestAppDelegate`, `TestFileExplorer`, `TestFlows`, `TestHorusAPI`, `TestPluginManager`, `TestRemotesManager`, `TestServer`, `TestWebAppMode`; Locust load tests (`Tests/TestServer/locust/`); mypy type-checking (`.mypy_cache/` baseline); `pylint` (`.pylintrc`); `Devtools/lint.sh`. **Core modules / files:**

- `Tests/` — all test modules
- `Devtools/lint.sh` — combined pylint + mypy run
- `.pylintrc`, `tsconfig.json` (TypeScript strict) **Hands-on:** Run `npm run test` (which invokes pytest) and `source Devtools/lint.sh`; fix a pylint warning in `Server/FlowManager/flow_manager.py`.

Open-Source Transition Roadmap

Phase 0 — Pre-Launch Hygiene (Weeks 1–3)

Actions to complete before the repository becomes public.

- **Secrets audit:** scan git history with `truffleHog / git secrets`; revoke any exposed tokens (CI `PYPI_API_TOKEN`, `GITHUB_TOKEN` references in CI, any hardcoded `debugPassword`).
- **License headers:** confirm `LICENSE.md` (repository) and `LICENSES.md` (third-party) are accurate and complete; add SPDX headers to all Python source files.
- **Scrub private infrastructure:** remove or replace the `perry` self-hosted runner references in `ci.yml`; do not expose internal hostnames (`MICROMAMBA_BINARY_PATH`, `DOCS_BUILD_DIR`) in the public repo.
- **Sanitise AppSupport artefacts:** `AppSupport/logs/`, `AppSupport/tmp/`, `Flows/` contain real PDB files and flow history — add them to `.gitignore` and remove from git history before going public.
- **Review `.github/copilot-instructions.md`:** ensure it contains no internal details.

Phase 1 — Community Infrastructure (Weeks 3–6)

- **Migrate CI to GitHub-hosted runners** (`ubuntu-latest`, `macos-latest`): replace `perry`-specific binary paths with `conda/mamba-action` and `setup-node`.
- **CONTRIBUTING.md:** document the fork-and-PR workflow, branch naming (`feat/`, `fix/`, `chore/`), commit message convention, and the two-approver merge policy.

- **Issue & PR templates:** promote the existing `.gitlab/issue_templates/` to `.github/ISSUE_TEMPLATE/` (bug report, feature request, improvement, docs).
- **Code of Conduct:** add `CODE_OF_CONDUCT.md` (Contributor Covenant).
- **Security policy:** add `SECURITY.md` with a private vulnerability disclosure email.
- **Automated docs deploy:** update `build_docs.sh` to push Sphinx HTML to GitHub Pages (e.g. `gh-pages` branch) instead of the private `perry` path.
- **Semantic versioning enforcement:** add `commitlint` + `release-please` GitHub Action to auto-draft release notes and bump `HorusAPI/_version.py` from conventional commits.

Phase 2 — Open-Source Sustainability (Weeks 6–12)

- **Plugin Registry:** publish the `AppSupport/DefaultPlugins/Horus/` plugin as a reference repository on GitHub; document the `plugin.meta` schema in the official docs as the stable public API contract for third-party plugins.
- **HorusAPI stability contract:** tag `HorusAPI` as the *only* stable public API surface; mark all `Server/`, `App/`, and `Views/` APIs as internal. Introduce `CHANGELOG.md` driven by `release-please`.
- **Multi-platform CI matrix:** extend CI to build and test on both `ubuntu-latest` and `macos-latest`; add a Windows smoke-test in `--server` mode (no GTK/Qt dependency).
- **Dependency pinning:** lock `Devtools/Environment/conda_horus.yaml` to exact versions; add `dependabot.yaml` for both Python (`pip`) and JS (`npm`) dependency updates.
- **horus.config.json example:** add a sanitised `horus.config.json.example` file to the repository root so new WebApp deployments have a documented starting point.
- **Plugin dev template repository:** create a `horus-plugin-template` GitHub template repository with a working `plugin.meta`, skeleton `PluginBlock`, a pre-configured `pytest` test, and a GitHub Action that validates `HorusAPI` compatibility on push.

Phase 3 — Long-Term Governance (Month 3+)

- **Maintainer handbook:** document the release process (tag → CI → PyPI + GitHub Release + Docs), the triage workflow, and the escalation path for security issues.
- **Feature flags for WebApp mode:** extract webapp-specific features (quotas, registration, email) behind a clearly documented configuration surface so the community can extend without touching core server code.
- **Extension API stabilisation:** formally document `HorusAPI/src/extensions.py` and `HorusAPI/src/molstar.py` as part of the stable plugin API; add typed stubs.
- **Deprecation policy:** adopt a two-minor-version deprecation cycle; use `warnings.warn(..., DeprecationWarning)` in `HorusAPI` for any API changes.
- **Community forum / Discussions:** enable GitHub Discussions; link from `README.md` and `CONTRIBUTING.md`.